

SA LEVEL UP COURSE – SEP 2025



Microservices Architecture (MSA) From Concepts to Practices

By DungHV6 – FHN.DCS

Course Structure

1) Day 1:

Architectural style evolution

Microservices – A Bird 's Eye View, DDD as an underpinning

2) Day 2:

MSA Concepts, Patterns, Principles, Best Practices, Anti-patterns

Platform, Frameworks, Tools and Techniques to realize MSA

3) Day 3:

-Implement MSA with 2 MSA platforms (Spring Platform, Kubernetes/Openshift)

-Microservices' role in AI Era

Day 1



Intentionally left blank

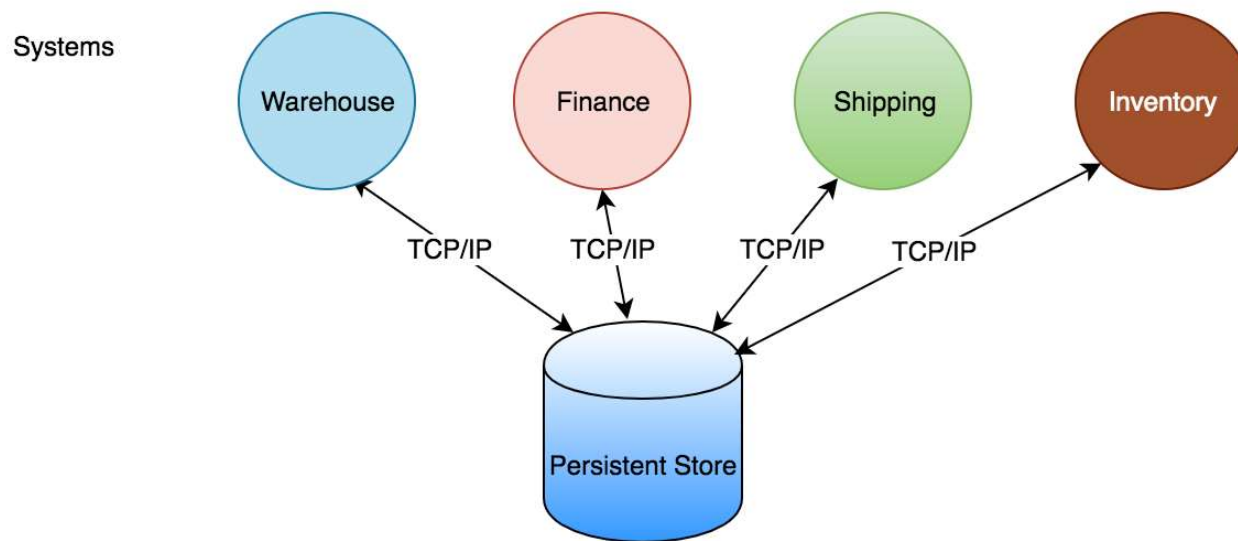
Architectural styles evolution – dating back



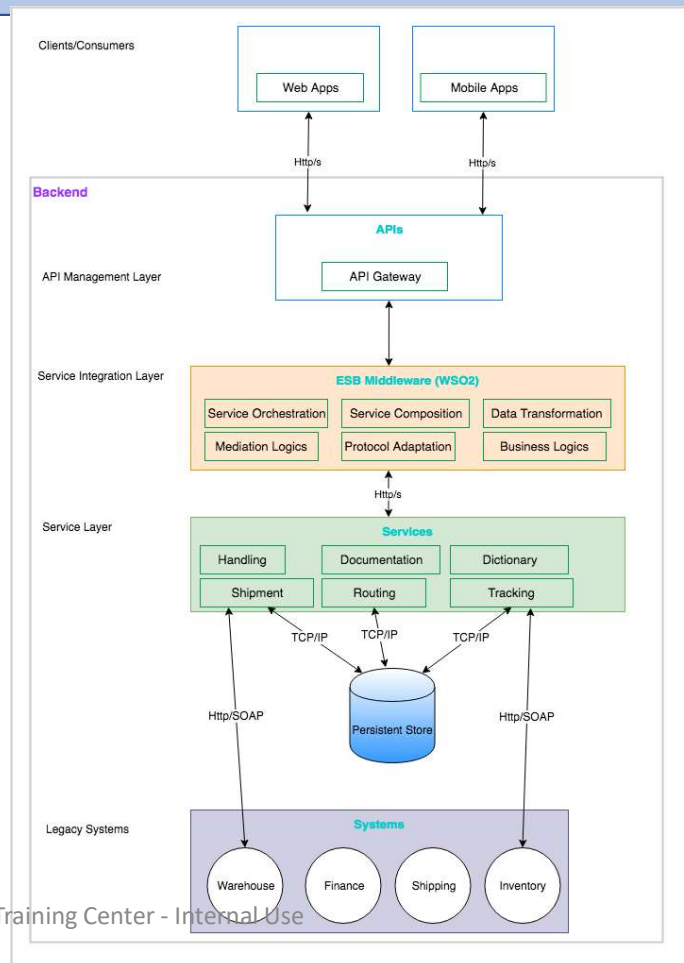
Architectural Styles for App Development:

- Multi-layered Monolithic Architecture (MA)
- Pipeline Architecture
- Microkernel (core system at the centre)
- Space-based Architecture
- Event-driven Architecture
- Orchestration-based Service-Oriented Architecture (SOA)
- Microservices Architecture (MSA)

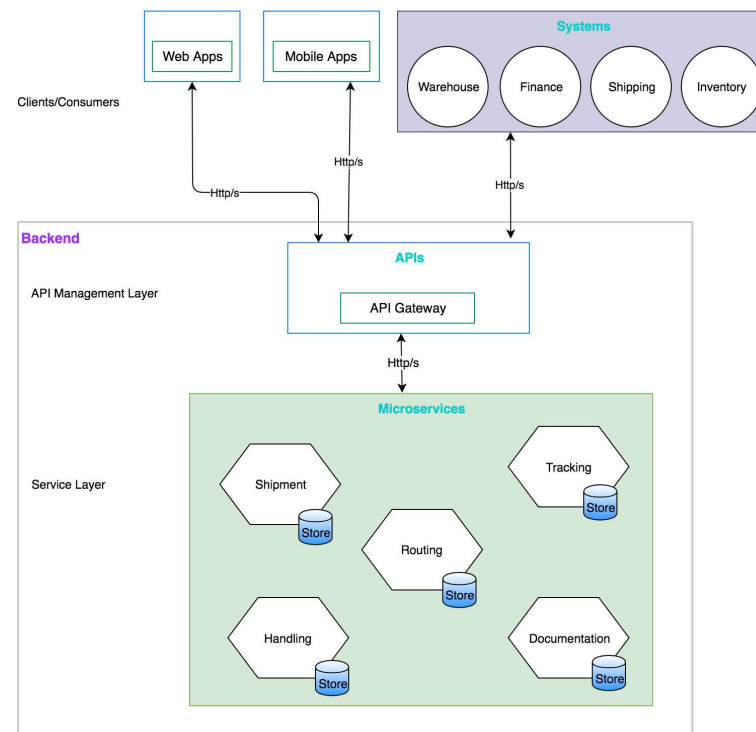
History – Monolithic Architecture (MA)



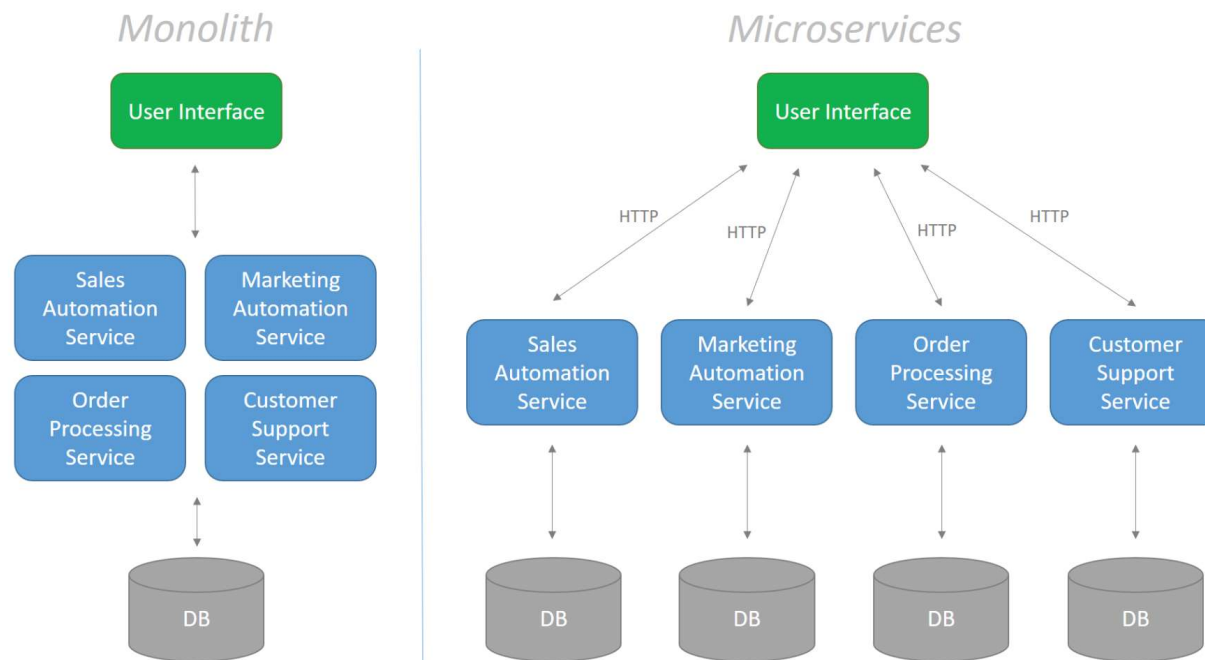
History – Service-Oriented Architecture (SOA)



Current – Microservices Architecture (MSA)



Monolith vs. SOA vs. Microservices



Source:

<https://www.carlippon.com/scalable-and-performant-asp-net-core-web-apis-microservices/>

SOA vs. MSA

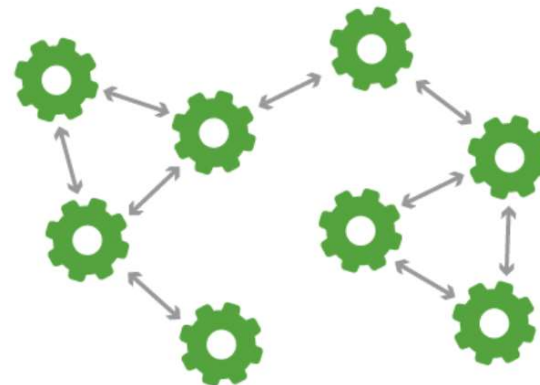
Similarities:

Both are service-oriented by splitting out a monolithic app into services. Data exchange among services occurs inside networks.

Differences:

SOA	MSA
Service orchestration often requires middleware and centralized integration logics (that span services, cause tight coupling)	No middleware and centralized integration point, lean on choreography
Applicable at enterprise-level, thus developed by units	Applicable at application-level, thus developed by teams in the same project
Flexibility by orchestration	Flexibility by fast development/deployment
Orchestration Logics in ESB & even Business Services are deployed in monoliths	Services are deployed in individual microservices, no orchestration middlewares

SOA vs. MSA (Cont.)



Source:

[https://dzone.com/articles/microservices-vs-soa-is-there-any-difference-at-al](https://dzone.com/articles/microservices-vs-soa-is-there-any-difference-at-all)

Microservices – Introduction in brief



- MSA is an architectural style in which an application composed of simple, manageable, autonomous, loosely design-time coupled, independently deployed components split out from a monolithic one and collaborate together.
- Each microservice is mapped to a business capability/functionality (business capability is something that a business does in order to generate value). They can be identified by analyzing the organization's purpose, structure, and business processes.
- Fulfill 12-factors

Microservices Definition



- MSA is an architectural style in which an application composed of simple, manageable, autonomous, loosely design-time coupled, independently deployed components split out from a monolithic one and collaborate together. (defined by Chris Richardson, world top Microservices thought leader).
- A system qualifies as as a MSA-based application when it adheres to MSA patterns and principles, fulfills 12-factor best practices, and avoid anti-patterns

Motivations behind MSA

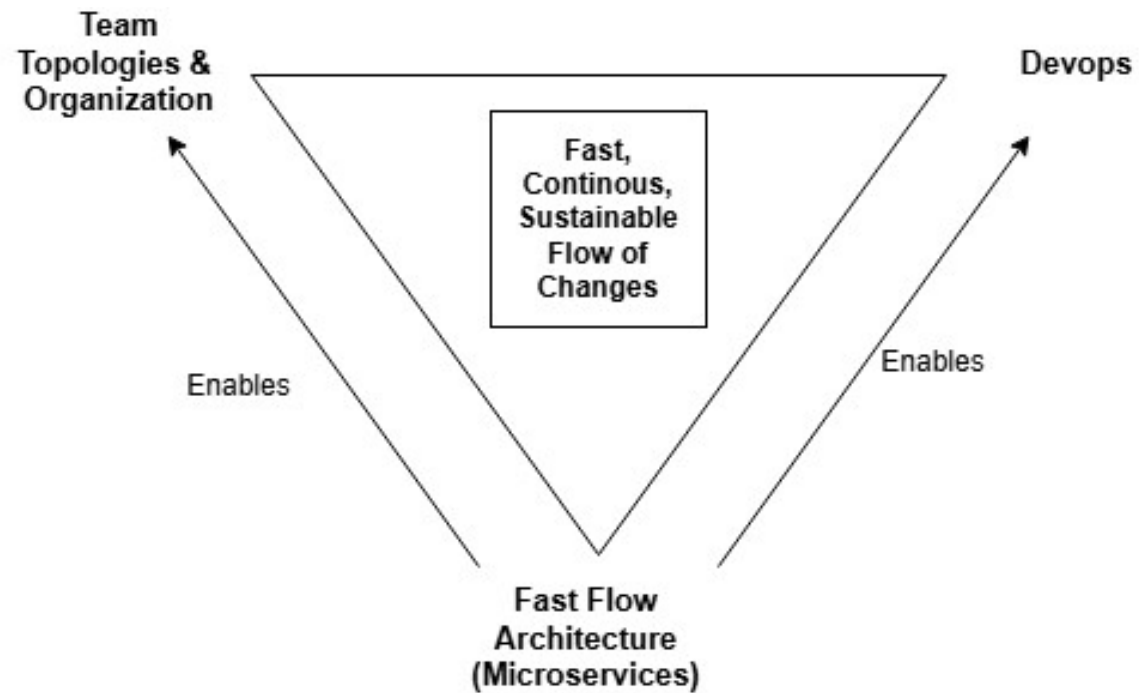


Motivations for Microservices:

- MSA overcomes limitations/painpoints of preceding SOA architectural style (tightly coupled integration logics centralized in monolithic ESB integration point for protocol and data format translations)
- MSA is considered as the best fast-flow architecture style that enables Team Topologies and Devops to achieve a fast/continuous/sustainable flow of changes in modern software development, which speeds up time-to-market of business ideas and customer feedbacks on them.

Motivations behind MSA (Cont.)

Fast Flow Triangle:



Why MSA?

MSA brings benefits as below:

- Services are small, independent and loosely coupled.
- Encapsulate a customer or business scenario.
- Apps are scalable and reusable.
- Easy for new developers to understand independent services.
- Easy to upgrade the framework of independent services.
- Each service has its own database.
- Easier to build & maintain & works great with Docker.
- Disaster Recovery is much easier to work with independent services than to deal with the whole system.
- Easy DevOps, short time to production

Benefits



- Independent, quick development, deployment, time-to-market
- Decoupling, thus, flexible to change, expand.
- Small, focused teams.
- Fault-tolerance, resilience.
- Selective and independent scalability.
- Technology heterogeneity for building microservices in an MSA-based app (Java, .NET, Python, Typescripts, Go, Ruby, RDBMS, NoSQL...)
- Facilitate DevOps

Challenges

MSA comes at a cost, a series of challenges posed as follows:

- Developing distributed systems can be complex(requests traveling, latency, ...).
- Multiple databases and transaction management can be painful.
- Testing a microservices-based application can be cumbersome.
- Deploying microservices can be complex.
- Logging, tracing and monitoring in distributed environment are burdens

Challenges (Cont.)



- Level of modularization, boundaries between subdomains
- Fault Management in distributed environment with Tracing/Logging/Monitoring
- Corporate culture
- Data Management
- Complex DevOps pipeline
- Management of Polyglot Architecture (heterogeneous development platforms, tech stacks, programming languages, database vendors...)
- Infra Provisioning
- Testing disparate microservices
- Governance

Strongly recommended:

Delegate the tackling of these challenges to frameworks/libraries (open-source or commercial)

Design Principles

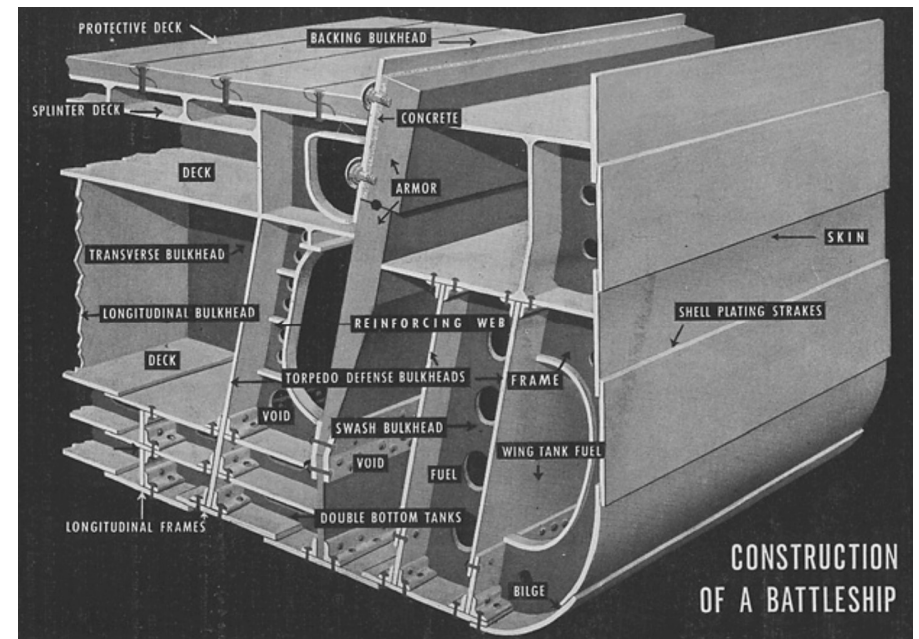
- Single Responsibility

Ideally, a single or few responsibilities (business capability or subdomains)

- Loose coupling

Ideally, a single microservice can fulfill a request, without communicating with another microservice, thus no dependencies, no cascading impacts exerted by business changes, expansion

Analogies – Honeycomb, Bulkheads

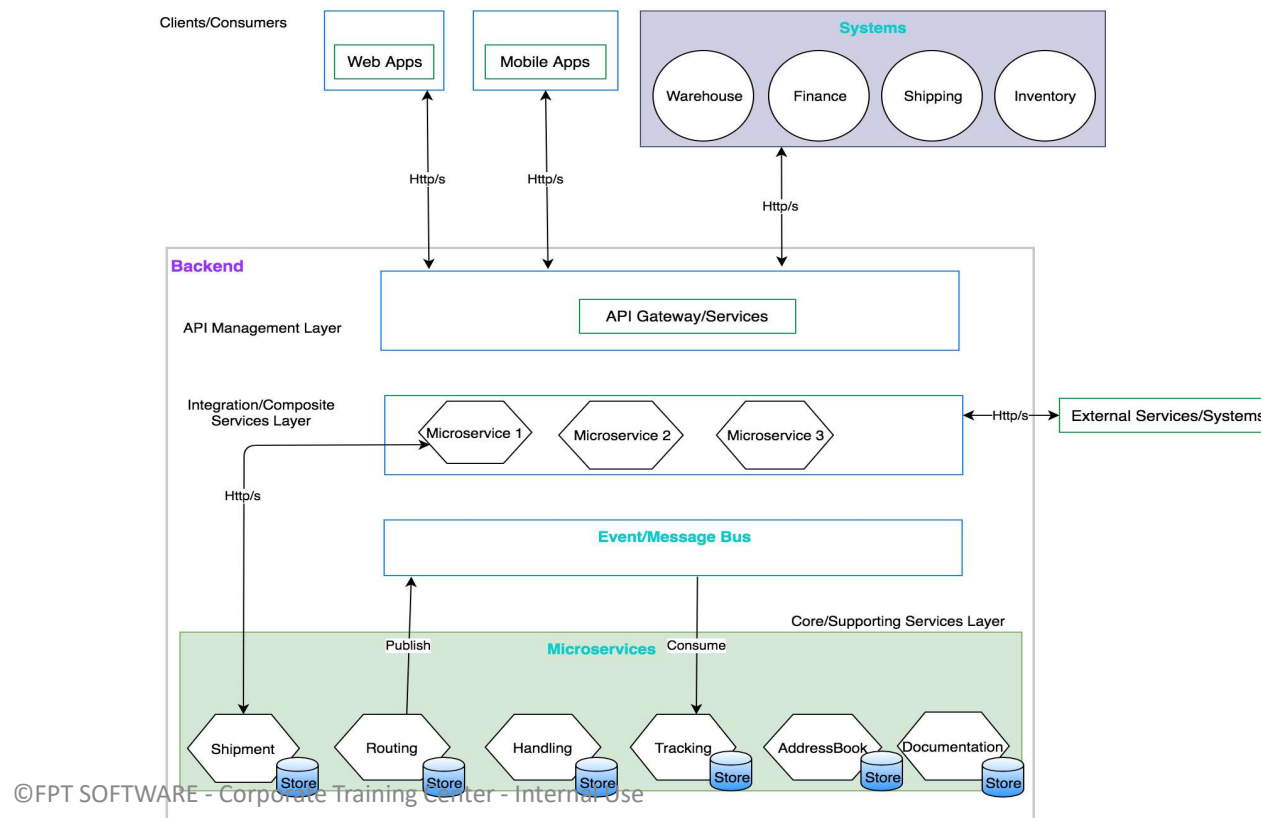


Source:

<https://www.localharvest.org/elk-river-apiary-M70399>

<https://www.ibiblio.org/hyperwar/USN/ref/Seamanship/Seamanship-4.html>

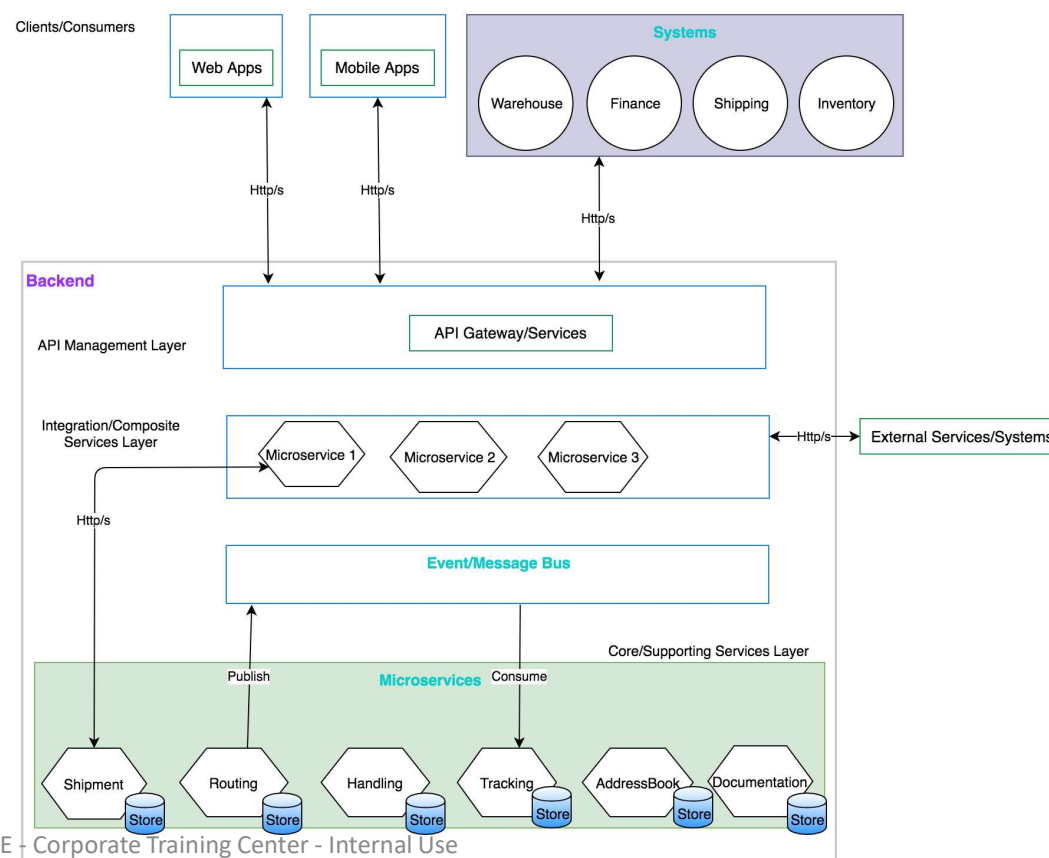
High-level Microservices Architecture



Types of Microservices

- Microservices fall into 3 main categories: core microservices, supporting/backing microservices, generic microservices
- **core microservices**
microservices implementing business capabilities/areas that differentiate a(n) organization/company from others, which brings competitive edges for it
- **supporting/backing microservices**
- generic microservices:

Types of Microservices (Cont.)



How to Achieve a Microservices-enabled App



- **Apply DDD approach, MSA patterns, 12-factor cloud app best practices and beyond 12 Factors (API First, Telemetry), avoid anti-patterns**
- MSA is business-driven. Decomposition of a monolithic application is based upon business capabilities.
- Technical engineers work in close collaboration with domain experts to split out the whole business domain into subdomains. Microservices correspond to these subdomains.
- A sample in logistics domain: Order, Shipping, Inventory in Logistics
- Reference: Domain-driven Design by Van

Domain-Driven Design (DDD) Approach



DDD is one of the approach to architecting software systems, with the aid of strategic and tactical patterns, tools and practices

Applied to both monolithic and microservices architectural styles
Destined for addressing the root cause of failed software projects (communication), tackling domains with high level of complexity

Business Domain, Subdomains



- **Business Domain**

main business area of a company, organization

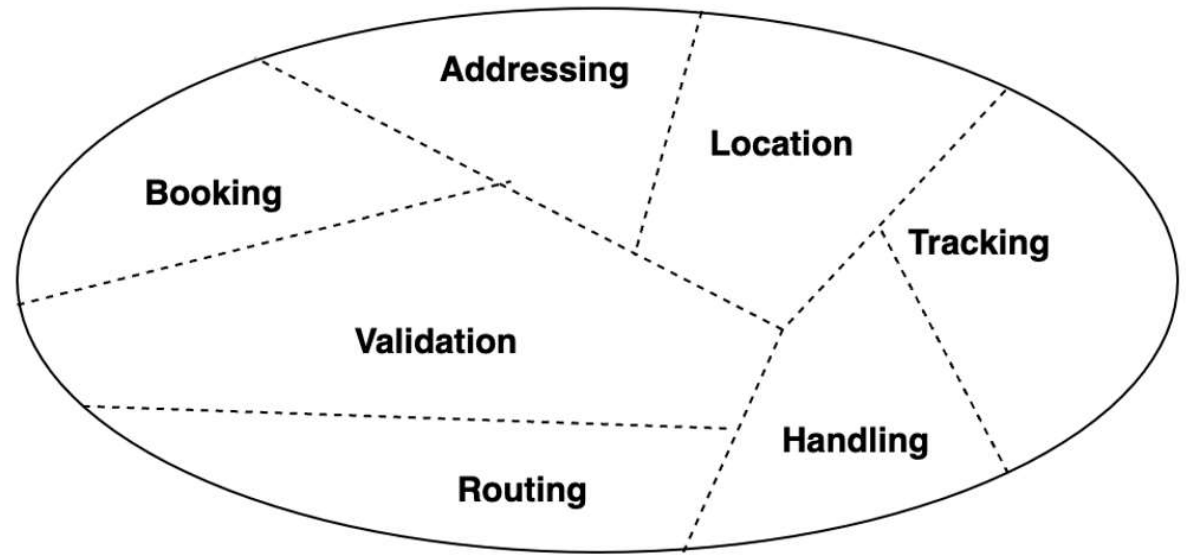
Examples:

FedEx, Uber, AirBnB, Walmart, Starbucks, Costco, Google, Microsoft, Amazon

- **Business Subdomain**

A subdomain: a fine-grained area of business activity, a service that a(n) company/organization provides to its customers/clients/end-users, a part that forms its whole business domain

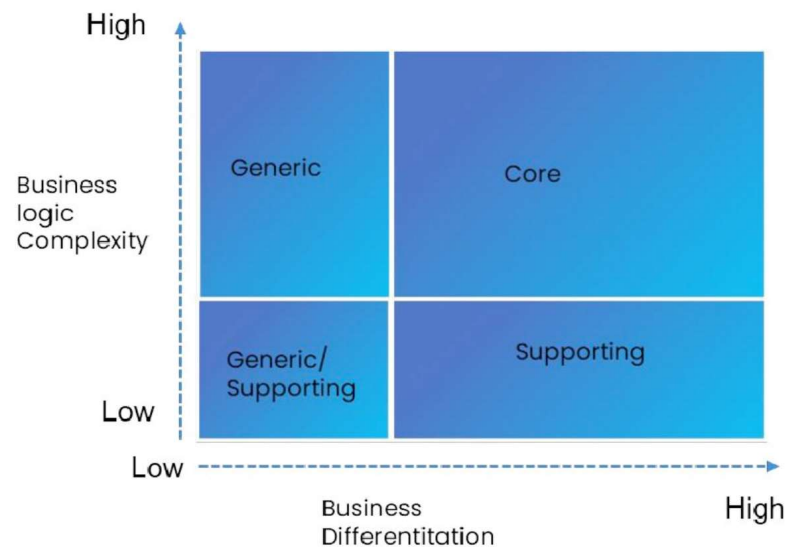
Sample Business Domain & Subdomains



Business Domain: Shipping

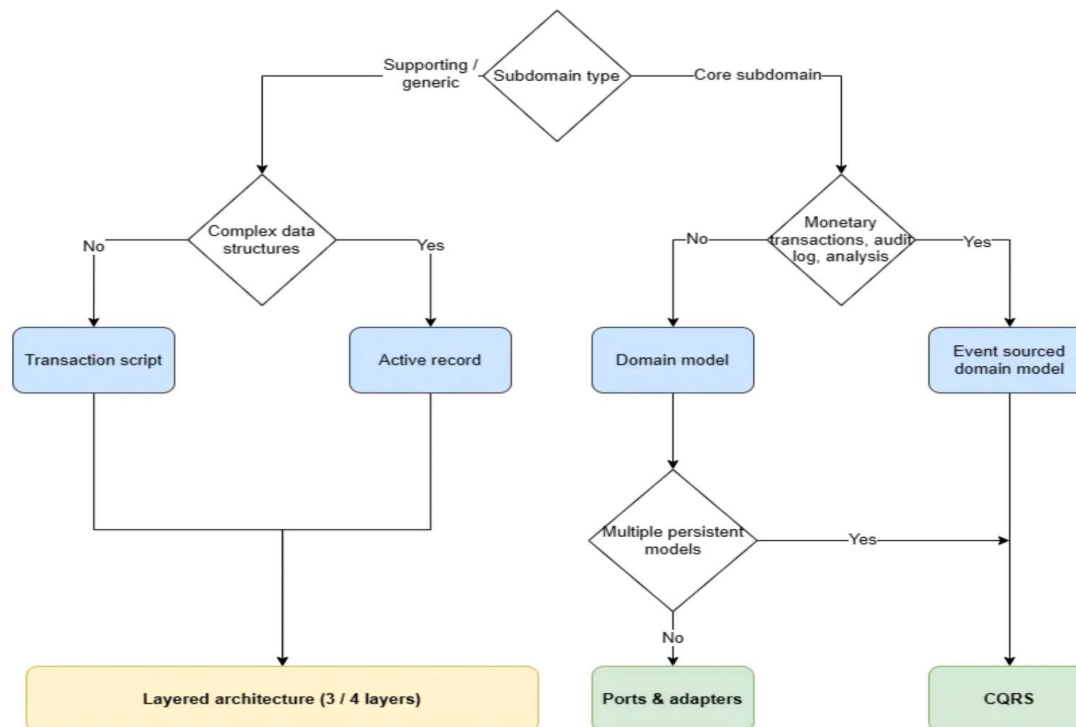
Subdomain Types

There exist 3 types of subdomains: Core, Supporting, Generic Subdomains



Source: speakerdeck.com

Decision Tree for business logic implementation pattern for Subdomains



Source: medium.com

Domain Decomposition Methods

1) Domain decomposition by business capabilities:

- Business capabilities/areas are services a(n) company/organization provides, which bring business values back to it.
- These business capabilities are grouped into categories and undertaken by business departments/units

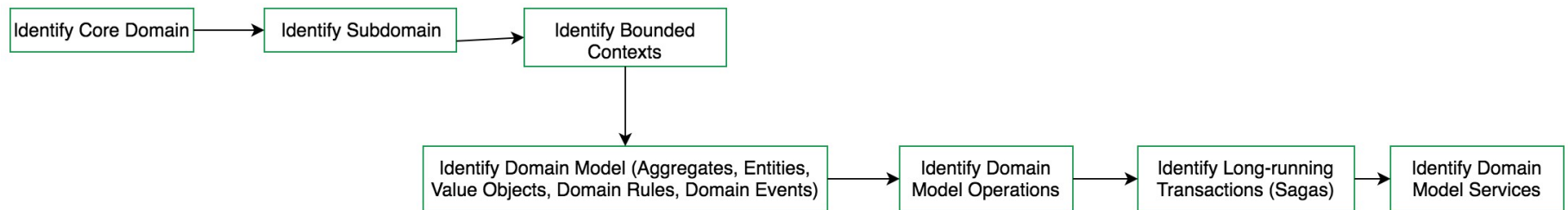
2) Decomposition by subdomains:

is the DDD approach focusing on modeling the business processes and flows

Business Capability

- A business capability is a service (without the aid of technologies) that generates business values to a(n) business/company/organization as well as end users
- How to identify business capabilities?
- Based on business 's purposes, organization structures, processes
- E.g: Insurance Industry: consumer 's application, claiming,
- Business Capability Features/Specs:
 - Revolve around a business object
 - Has inputs/outputs
 - Remain stable while the manner it provides (or the form it takes) changes drastically
 - Iterative for clarity, core of business capability hardly change

DDD – Domain Modeling Process



DDD – Domain Modeling Process (Cont.)



- Design app with Domain-Driven Design (DDD) as Design Approach
- Identify problem space or business domain
- Decompose the business domain into subdomains
- Map subdomains to bounded contexts, or BCs (as solutions to subdomains)
- Create BC 's domain model consisting of DDD artifacts (Entities, Value Objects, Business Rules, Domain Events, Sagas,
- Domain Operations including queries, commands)
- Domain Model is the implementation of the core business logics inside a Bounded Context.

Identify Subdomains

The decomposition of business domain into subdomains is based on:

- From business perspective, business capabilities/areas, provided by a company/organization (without the aid of software). A business capability brings business value, financial return, customer value delivery, taken responsibility by a department and helps run and operate a company.
- From technical perspective, adherence to Single Responsibility Principle (SRP) for loosely coupling of bounded contexts and cohesion of a bounded context.

Identify Bounded Contexts (BCs)



A Bounded Context (BC) is a system that fulfills the goals of the business in the real world and a solution to a subdomain

Context Mapping

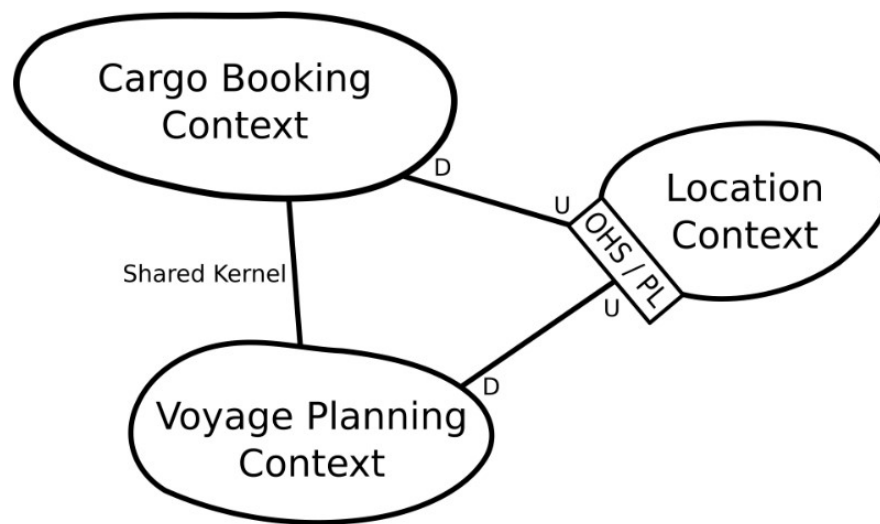
Context Mapping represents the relationships among bounded contexts (upstream, downstream, dependencies...)

Relationship Types:

Anti-corruption Layer, Shared Kernel, Open Host Service, Customer/Supplier, Conformist...

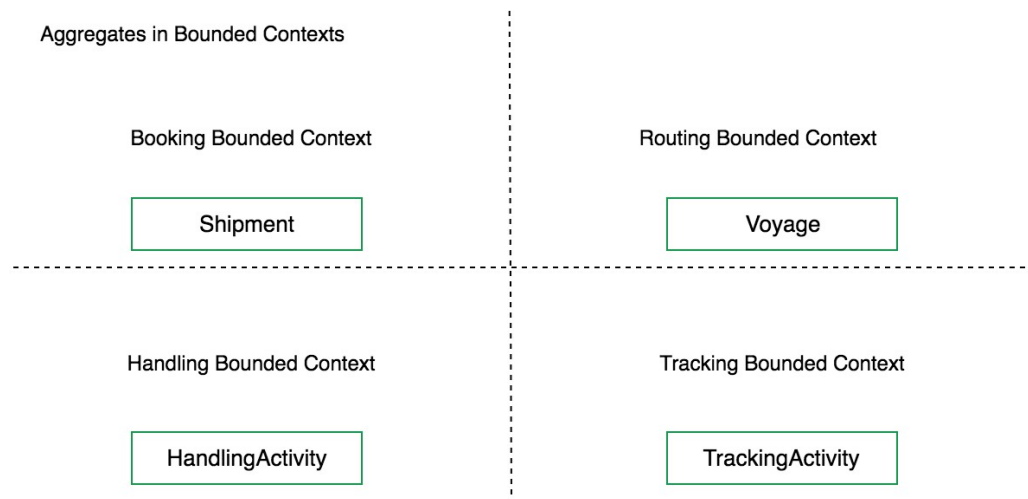
Ref: <https://medium.com/ingeniouslysimple/context-mapping-in-domain-driven-design-9063465d2eb8>

Sample Context Map



Source: <https://contextmapper.org/docs/examples/>

Identify Domain Models for Bounded Contexts



- Each BC maps out to a microservice
- Microservices fall under 3 categories: Core Services, Supporting Services (including integration and composite services) and Generic Services

Domain Model - DDD Artifacts (inside Problem Space)



- Business Entities
- Business Rules
- Business Flows
- Business Operations
- Business Events

Domain Model - Technical Artifacts (inside Solution Space)



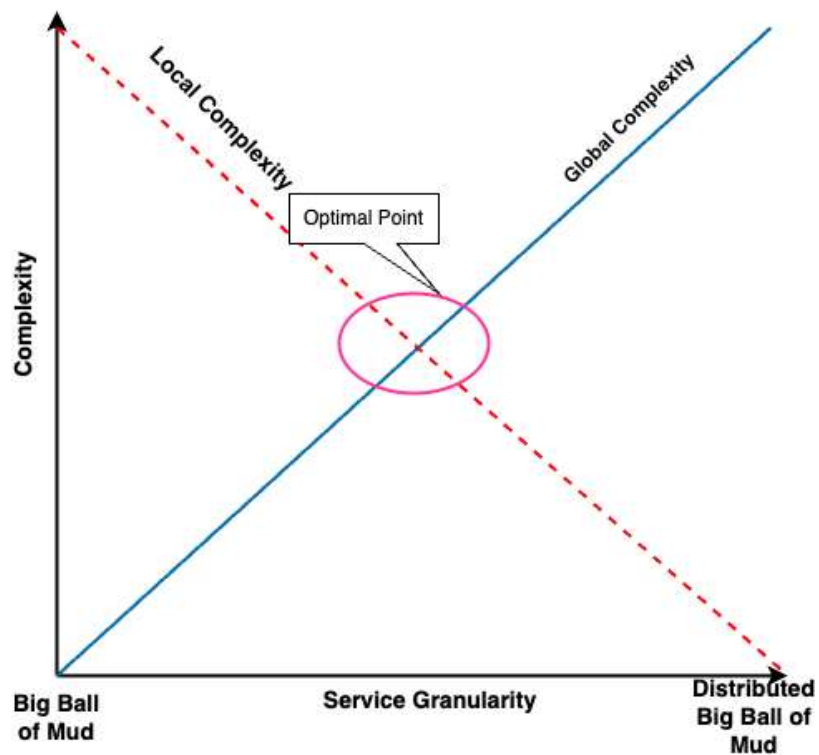
- Aggregate Roots/Entities/Value Objects (VOs)
- Domain Rules & Domain Services
- Sagas
- Commands/Queries
- Domain Events

Notes on Domain Decomposition

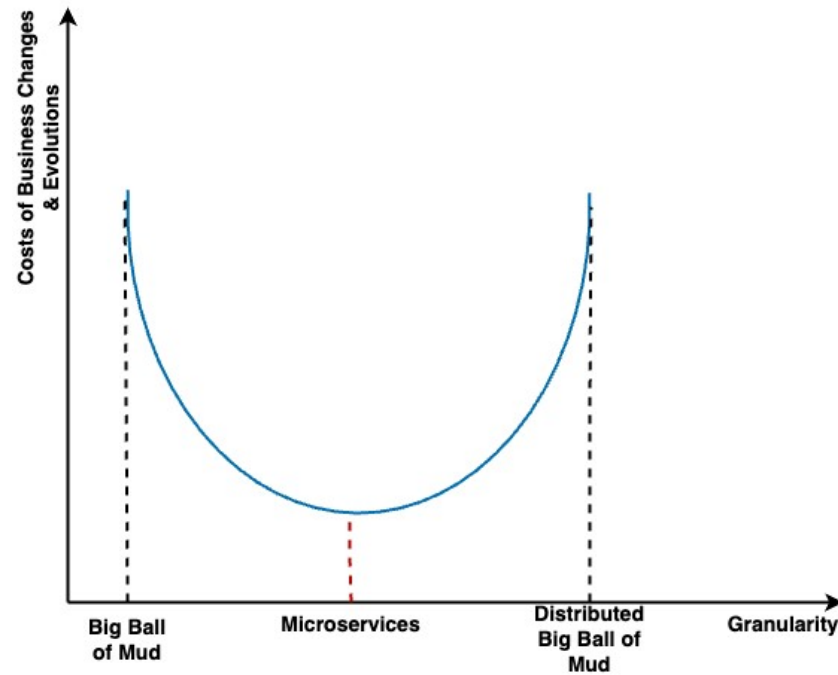


- The service decomposition (the decisive factor) starts from business view (specially from customer side) and in collaboration w/ technical view.
- A bounded context 's granularity should not taken to extremes (too fine-grained or coarse-grained), at data layer, a group of around 4 or 5 database tables is a good fit.
- The size of a microservice is not determined by lines of code, but the business logics of a subdomain, so that changes inside one subdomain do not cascade to others, also applicable to redeployment.
- A business capability/area that needs scalability for high loads is eligible for a bounded context or a microservices, for instance, booking subdomain.
- As a matter of fact, like in Monolith, the business requirements can not be clear from the outset, and service decomposition often undergo reiterations further refinements as the project progresses.
- Migration follow Strangle pattern, prioritize migration of core services first

Granularity & Local/Global Complexities



Granularity & Costs of Changes



Domain Decomposition– Hands-on



Decompose an Online Coffee Shop as the whole domain into corresponding subdomains, respectively, based on 2 afore-mentioned decomposition approaches.

Implement DDD Artifacts



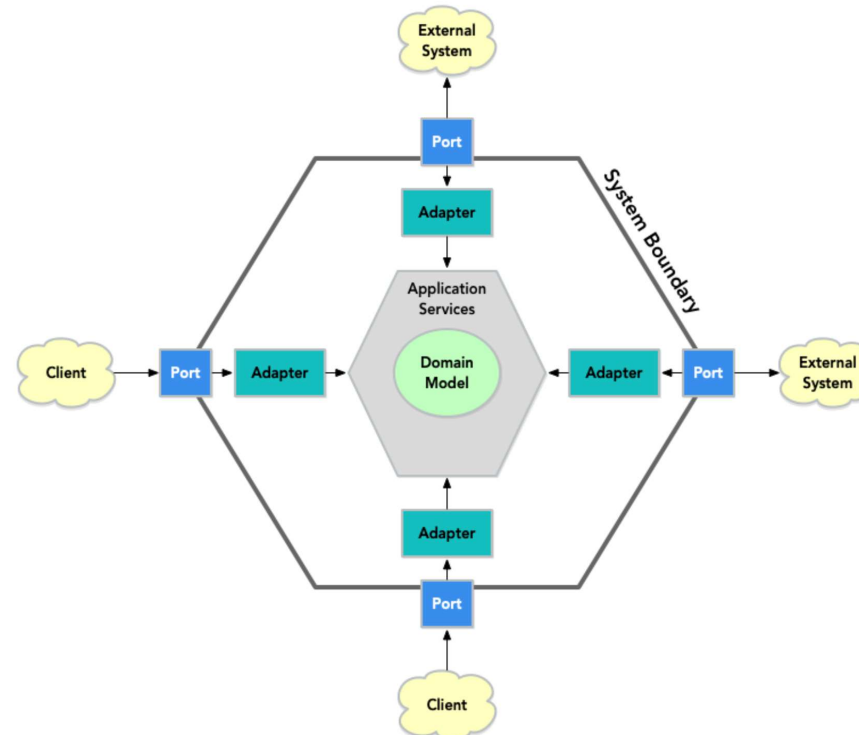
- Choose a technology stack (Java Spring ,for instance, to build a microservice), a Microservice platform (Kubernetes/Openshift, Spring Cloud, for instance) to implement DDD artifacts, apply MSA patterns and 12-factor app manifesto, adhering to Hexagonal Architecture (Slide 50)

Internal Code Structure of a Microservice



- Internal code structure of a microservice corresponds to DDD artifacts

Internal Code Structure (Cont.)



Source: <https://vaadin.com/learn/tutorials/ddd/ddd> and hexagonal

Internal Code Structure – Hands-on



Lean on your tech stack (Spring stack highly recommended), create microservice code structure implementing DDD artifacts and in compliance with Hexagonal Architecture

References

MSA Topics:

Domain Driven Design, Microservices Patterns,

Must-read Web Pages:

<https://microservices.io>

Recommended Books:

- Implementing Domain Driven Design
- Patterns, Principles and Practices of Domain-Driven Design (strongly recommended)
- Tackle complexity
- Microservices Patterns
- Spring Microservice in action
- Kubernetes/Openshift in action
- Istio in action
- Microservices in .NET

Thank you for joining us!

